

05_prompttemplates 大纲

1.提示词模板(Prompt Templates)	1
1.1 复习: str.format()	1
1) 带有位置参数的用法	2
2) 带有关键字参数的用法	2
3) 使用字典解包的方式	2
4) 字符串拼接方式	2
1.2 提示词模板	3
1) PromptTemplate --langchain1.0之后弱化了	3
(1) 两种实例化方式	4
方式1: 构造方法 format()	4
举例1: 单变量	4
举例2: 定义多变量	4
方式2: 调用from_template() 推荐!!!	4
(2) 两种新的结构形式	5
形式1: 部分提示词模版	5
方式1: 实例化过程中使用partial_variables变量	5
方式2: 使用 PromptTemplate.partial() 方法创建部分提示模板	5
形式2: 组合提示词(了解)	6
(3) 给变量赋值的两种方式: format() 与 invoke()	7
(4) 结合LLM调用	8
2) ChatPromptTemplate	8
(1) 两种实例化方式	9
方式1: 使用实例初始化方法	9
方式2: 调用from_messages()--推荐	10
(2) 调用提示词模板的几种方法	10
方法1: invoke() ---返回ChatPromptValue	10
方法2: format() ---返回str	11
方法3: format_messages() ---返回消息构成的list	11
方法4: format_prompt() ---返回ChatPromptValue	11
如何实现ChatPromptValue与list[messages]、字符串之间转换?	12
(3) 更丰富的实例化参数类型	13
举例1: 元组列表	13
举例2: 字符串列表	14
举例3: 字典列表	14
举例4: 消息对象列表	15
举例5: BaseChatPromptTemplate参数列表	16
举例6: BaseMessagePromptTemplate参数列表	16
综合案例	17
(4) 结合LLM	18
(5) ChatPromptTemplate的高级特性	19
1) 部分变量的预填充: partial()	19
2) 消息占位符	20

JSON形式	20
插入消息列表: MessagesPlaceholder	21
3) 存储对话历史内容	22
4) 可复用的模板库	23
举例1: template.py文件声明模板库	23
举例2: 分类创建模板	24
5) 模板组合	24
方法1: 字符串组合	24
方法2: 使用+运算符	25
1.3 少量样本示例的提示词模板	25
1) 使用说明	25
2) FewShotPromptTemplate使用	25
举例1 未提供示例的情况	25
举例2 使用FewShotPromptTemplate	26
3) FewshotChatMessagePromptTemplate的使用	27
4) Example selectors(示例选择器)	29
举例1: SemanticSimilarityExampleSelector	29
举例2: 结合FewShotPromptTemplate使用	31
5) PipelinePromptTemplate(了解)	32
6) 自定义提示词模版(了解)	34
7) 从文档中加载Prompt(了解)	35
(1) yaml格式提示词	35
(2) json格式提示词	35
(3) 调用py文件提示词模板	36

1.提示词模板(Prompt Templates)

Prompt Template，通过模板管理大模型的输入。

Prompt Template 是LangChain中的一个概念，接收用户输入，返回一个传递给LLM的信息（即提示词prompt）。在应用开发中，固定的提示词限制了模型的灵活性和适用范围。所以，prompt template 是一个模板化的字符串，你可以将变量插入到模板中，从而创建出不同的提示。

调用时：以字典作为输入，其中每个键代表要填充的提示模板中的变量。输出一个 PromptValue。这个 PromptValue 可以传递给 LLM 或 ChatModel，并且还可以转换为字符串或消息列表。

有几种不同类型的提示模板：

- PromptTemplate：LLM提示模板，用于生成字符串提示。它使用 Python 的字符串来模板提示。
- ChatPromptTemplate：聊天提示模板，用于组合各种角色的消息模板，传入聊天模型。
- XxxMessagePromptTemplate：消息模板词模板，包括：
 - SystemMessagePromptTemplate、
 - HumanMessagePromptTemplate、AIMessagePromptTemplate、
 - ChatMessagePromptTemplate等
- FewShotPromptTemplate：样本提示词模板，通过示例来教模型如何回答
- PipelinePrompt：管道提示词模板，用于把几个提示词组合在一起使用。
- 自定义模板：允许基于其它模板类来定制自己的提示词模板。

```
In [ ]: from langchain_core.prompts import PromptTemplate
        from langchain_core.prompts import ChatPromptTemplate
        from langchain_core.prompts import FewShotPromptTemplate
        from langchain_core.prompts import (
            ChatMessagePromptTemplate,
            SystemMessagePromptTemplate,
            AIMessagePromptTemplate,
            HumanMessagePromptTemplate,
        )
```

1.1 复习：str.format()

Python的str.format() 方法是一种字符串格式化的手段，允许在字符串中插入变量。

使用这种方法，可以创建包含占位符的字符串模板，占位符由花括号{}标识。调用format()方法时，可以传入一个或多个参数，这些参数将被顺序替换进占位符中。str.format()提供了灵活的方式来构造字符串，支持多种格式化选项。

在LangChain的默认设置下， PromptTemplate 使用 Python 的str.format() 方法进行模板化。这样 在模型接收输入前，可以根据需要对数据进行预处理和结构化。

1) 带有位置参数的用法:

```
In [1]: # 使用位置参数
info = "Name: {0}, Age: {1}".format("Jerry", 25)
print(info)
```

Name: Jerry, Age: 25

2) 带有关键字参数的用法:

```
In [2]: # 使用关键字参数
info = "Name: {name}, Age: {age}".format(name="Tom", age=25)
print(info)
```

Name: Tom, Age: 25

3) 使用字典解包的方式:

```
In [3]: # 使用字典解包
person = {"name": "David", "age": 40}
info = "Name: {name}, Age: {age}".format(**person)
print(info)
```

Name: David, Age: 40

4) 字符串拼接方式:

- 优点：简单，适合临时demo，无学习成本
- 可读性差，不易维护，无变量校验，难以支持复杂场景

```
In [4]: from langchain.chat_models import init_chat_model
import os
from dotenv import load_dotenv

load_dotenv()

model = init_chat_model(
    model="deepseek:deepseek-v4-flash",
    api_key=os.getenv("DEEPSEEK_API_KEY"),
    base_url=os.getenv("DEEPSEEK_BASE_URL"),
    # extra_body={"thinking":{"type":"enabled"}}
)
```

```
In [5]: # 字符串拼接
topic = "Python"
difficulty = "初学者"
```

```
# 难以维护容易出错
prompt_str = f"你是一个{difficulty}级别的编程导师，请用一句简单易懂的语言描述一下{topic}。"

response = model.invoke(prompt_str)
print(f"AI回复：{response.content}")
```

AI回复：Python 就像是一本“给电脑的魔法指令手册”，你用它写下人类能看懂的命令，电脑就能帮你自动完成各种任务。

1.2 提示词模板

1) PromptTemplate --langchain1.0之后弱化了

输出：单个字符串

只有一段文本，没有区分角色（system/user/assistant）

```
In [1]: from langchain_core.prompts import PromptTemplate

# 字符串拼接
topic = "Python"
difficulty = "初学者"

# 难以维护容易出错
template = PromptTemplate.from_template(
    "你是一个{difficulty}级别的编程导师，请用一句简单易懂的语言描述一下{topic}。"
)

prompt = template.format(difficulty=difficulty,topic=topic)
print(prompt) # 返回字符串
```

你是一个初学者级别的编程导师，请用一句简单易懂的语言描述一下Python。

PromptTemplate类，用于快速构建包含变量的提示词模板，并通过传入不同的参数值生成自定义的提示词。

- PromptTemplate如何获取实例（掌握两种方式：.format() .from_template()）
- 两种特殊结构的使用（部分提示词模板的使用、组合提示词的使用）
- 给变量赋值的两种方式(掌握 format () invoke ())
- 结合大模型的使用

主要参数介绍：

template：定义提示词模板的字符串，其中包含文本和变量占位符（如{name}）；

input_variables：列表，指定了模板中使用的变量名称，在调用模板时被替换；

partial_variables：字典，用于定义模板中一些固定的变量名。这些值不需要再每次调用时被替换。

函数介绍：

format()：给input_variables变量赋值，并返回提示词。利用format() 进行格式化时就一定要赋值，否则会报错。当在template中未设置input_variables，则会自动忽略。

(1) 两种实例化方式

方式1：构造方法 format()

举例1：单变量

```
In [2]: from langchain_core.prompts import PromptTemplate
# 定义模板：描述主题的应用
template = PromptTemplate(template="请简要描述{topic}的应用。",
                           input_variables=["topic"])
print(template)

# 使用模板生成提示词
prompt_1 = template.format(topic="机器学习")
prompt_2 = template.format(topic="自然语言处理")

print("提示词1:", prompt_1)
print("提示词2:", prompt_2)
```

input_variables=['topic'] input_types={} partial_variables={} template='请简要描述{topic}的应用。'

提示词1: 请简要描述机器学习的应用。

提示词2: 请简要描述自然语言处理的应用。

可以直观的看到PromptTemplate可以将template中声明的变量topic准确提取出来，使prompt更清晰。

举例2：定义多变量

```
In [ ]: from langchain_core.prompts import PromptTemplate
#定义多变量模板
template = PromptTemplate(
    template="请评价{product}的优缺点，包括{aspect1}和{aspect2}。",
    input_variables=["product", "aspect1", "aspect2"])
#使用模板生成提示词
prompt_1 = template.format(product="智能手机", aspect1="电池续航", aspect2="拍照质量")
prompt_2 = template.format(product="笔记本电脑", aspect1="处理速度", aspect2="便携性")

print("提示词1:", prompt_1)
print("提示词2:", prompt_2)
```

方式2：调用from_template() 推荐！！

```
In [ ]: from langchain_core.prompts import PromptTemplate

prompt_template = PromptTemplate.from_template(
    "请给我一个关于{topic}的{type}解释。"
)
```

```
#传入模板中的变量名
prompt = prompt_template.format(type="详细", topic="量子力学")

print(prompt)
```

模板支持任意数量的变量，包括不含变量：

```
In [3]: #1. 导入相关的包
from langchain_core.prompts import PromptTemplate
# 2. 定义提示词模版对象
text = """
Tell me a joke
"""

prompt_template = PromptTemplate.from_template(text)

# 3. 默认使用f-string进行格式化（返回格式好的字符串）
prompt = prompt_template.format()
print(prompt)
```

Tell me a joke

（2）两种新的结构形式

形式1：部分提示词模版

方式1：实例化过程中使用partial_variables变量

```
In [ ]: from langchain_core.prompts import PromptTemplate
#方式2：
template2 = PromptTemplate(
    template="{foo}{bar}",
    input_variables=["foo", "bar"],
    partial_variables={"foo": "hello"}#将部分变量初始化
)
prompt2 = template2.format(bar="world")

print(prompt2)
```

方式2：使用 PromptTemplate.partial() 方法创建部分提示模板

举例1：

```
In [5]: from langchain_core.prompts import PromptTemplate
template1 = PromptTemplate(
    template="{foo}{bar}",
    input_variables=["foo", "bar"]
)

#方式1：
partial_template1 = template1.partial(foo="hello")#.partial()方法必须返回值，它不会改变'ten
```

```
prompt1 = partial_template1.format(bar="world")
print(prompt1)
```

helloworld

举例2：直接在实例后面.partial()

```
In [4]: from langchain_core.prompts import PromptTemplate
# 完整模板
full_template = """你是一个{role}，请用{style}风格回答：
问题：{question}
答案："""

# 预填充角色和风格
partial_template = PromptTemplate.from_template(full_template).partial(
    role="资深厨师",
    style="专业但幽默"
) # 直接在实例后面用.partial()方法预填充变量

# 只需提供剩余变量
print(partial_template.format(question="如何煎牛排？"))
```

你是一个资深厨师，请用专业但幽默风格回答：
问题：如何煎牛排？
答案：

举例3：

```
In [6]: prompt_template = PromptTemplate.from_template(
    template = "请评价{product}的优缺点，包括{aspect1}和{aspect2}。",
    partial_variables = {"aspect1": "电池", "aspect2": "屏幕"}
)

prompt = prompt_template.format(product="笔记本电脑")
print(prompt)
```

请评价笔记本电脑的优缺点，包括电池和屏幕。

形式2：组合提示词(了解)

举例：

```
In [7]: from langchain_core.prompts import PromptTemplate
template = (
    PromptTemplate.from_template("Tell me a joke about {topic}")
    + ", make it funny"
    + "\n\nand in {language}"
)

prompt = template.format(topic="sports", language="spanish")
print(prompt)
```

Tell me a joke about sports, make it funny
and in spanish

(3) 给变量赋值的两种方式：format() 与 invoke()

只要对象是RunnableSerializable接口类型，都可以使用invoke()，替换前面使用format()的调用方式。

format()，返回值为字符串类型；

invoke()，返回值为PromptValue类型，接着调用to_string()返回字符串。

举例1：调用invoke () 参数是字典

```
In [8]: from langchain_core.prompts import PromptTemplate
#定义多变量模板
template = PromptTemplate(
    template="请评价{product}的优缺点，包括{aspect1}和{aspect2}。",
    input_variables=["product", "aspect1", "aspect2"])
#使用模板生成提示词
#prompt_1 = template.format(product="智能手机", aspect1="电池续航", aspect2="拍照质量")
prompt_1 = template.invoke({"product": "智能手机", "aspect1": "电池续航", "aspect2": "拍照质量"})
print(prompt_1)
print(type(prompt_1))
```

```
text='请评价智能手机的优缺点，包括电池续航和拍照质量。'
<class 'langchain_core.prompts.StringPromptValue'>
```

```
In [9]: #1. 导入相关的包
from langchain_core.prompts import PromptTemplate

# 2. 定义提示词模版对象
prompt_template = PromptTemplate.from_template(
    "Tell me a {adjective} joke about {content}."
)

# 3. 默认使用f-string进行格式化 (返回格式好的字符串)
prompt_template.invoke({"adjective": "funny", "content": "chickens"})
```

```
Out[9]: StringPromptValue(text='Tell me a funny joke about chickens.')
```

举例2：

```
In [10]: #1. 导入相关的包
from langchain_core.prompts import PromptTemplate

# 2. 使用初始化器进行实例化
prompt = PromptTemplate(
    input_variables=["adjective", "content"],
    template="Tell me a {adjective} joke about {content}")

# 3. PromptTemplate底层是RunnableSerializable接口 所以可以直接使用invoke() 调用
prompt.invoke({"adjective": "funny", "content": "chickens"})
```

```
Out[10]: StringPromptValue(text='Tell me a funny joke about chickens.')
```

举例3：

```
In [11]: from langchain_core.prompts import PromptTemplate

prompt_template = (
    PromptTemplate.from_template("Tell me a joke about {topic}")
    + ", make it funny"
    + " and in {language}"
)

prompt = prompt_template.invoke({"topic": "sports", "language": "spanish"})
print(prompt)
```

text='Tell me a joke about sports, make it funny and in spanish'

(4) 结合LLM调用

因为PromptTemplate()输出：单个字符串

只有一段文本，没有区分角色（system/user/assistant）

一般使用OpenAI()，现在主流的都用ChatOpenAI()当然ChatOpenAI()也可以接收普通字符串（内部会自动包装成一条 user message）OpenAI()只能接收字符串，不支持消息数组

2) ChatPromptTemplate

输出：消息对象列表 [SystemMessage, HumanMessage, AIMessage]

严格区分角色，贴合对话模型原生输入格式

```
In [ ]: from langchain_core.prompts import ChatPromptTemplate

prompt_template = ChatPromptTemplate([
    ("system", "你是一个AI开发工程师，你的名字是{name}"),
    ("human", "{user_input}")
])

prompt = prompt_template.invoke({"name": "Victor", "user_input": "你能帮我做什么?"})
print(prompt) # 返回消息列表
```

```
messages=[SystemMessage(content='你是一个AI开发工程师，你的名字是Victor', additional_kwargs={}, response_metadata={}), HumanMessage(content='你能帮我做什么?', additional_kwargs={}, response_metadata={})]
```

- 1、实例化的方式（两种方式：使用构造方法，from_messages()）
- 2、调用提示词模板的几种方法：invoke()\format()\format_messages()\format_prompt()
- 3、更丰富的实例化参数类型
- 4、结合LLM
- 5、插入消息列表：MessagePlaceholder

ChatPromptTemplate是创建聊天消息列表的提示模板。它比普通 PromptTemplate 更适合处理多角色、多轮次的对话场景。

特点：

支持 System / Human / AI 等不同角色的消息模板

对话历史维护

参数类型：列表参数格式是tuple类型（role:str content:str 组合最常用）

元组的格式为：

(role: str | type, content: str | list[dict] | list[object])

其中 role 是：字符串（如 "system"、"user"、"assistant"）

（1）两种实例化方式

方式1：使用实例初始化方法

举例：

```
In [12]: from langchain_core.prompts import ChatPromptTemplate
```

```
#创建实例
```

```
chat_prompt_template = ChatPromptTemplate([
    ("system", "你是一个AI助手. 你的名字是 {name}.",),
    ("human", "你好，最近怎么样",),
    ("ai", "我很好，谢谢",),
    ("human", "我的问题是:{question}"),])
```

```
chat_prompt = chat_prompt_template.invoke(
    {"name": "小谷AI", "question": "如何学习人工智能?"}
)
print(chat_prompt)
```

```
messages=[SystemMessage(content='你是一个AI助手. 你的名字是 小谷AI.', additional_kwargs={}, response_metadata={}), HumanMessage(content='你好，最近怎么样', additional_kwargs={}, response_metadata={}), AIMessage(content='我很好，谢谢', additional_kwargs={}, response_metadata={}, tool_calls=[], invalid_tool_calls=[]), HumanMessage(content='我的问题是:如何学习人工智能?', additional_kwargs={}, response_metadata={})]
```

```
In [13]: from langchain_core.prompts import ChatPromptTemplate
```

```
#参数类型这里使用的是tuple构成的list
```

```
prompt_template = ChatPromptTemplate([
    # 字符串 role + 字符串 content
    ("system", "你是一个AI开发工程师. 你的名字是 {name}.",),
    ("human", "你能开发哪些AI应用?",),
    ("ai", "我能开发很多AI应用, 比如聊天机器人, 图像识别, 自然语言处理等."),
    ("human", "{user_input}")
])
```

```
#调用invoke()方法，返回字符串
```

```
prompt = prompt_template.invoke(input={"name": "小谷AI", "user_input": "你能帮我做什么?"})
```

```
print(type(prompt))
print(prompt)
```

```
<class 'langchain_core.prompt_values.ChatPromptValue'>
messages=[SystemMessage(content='你是一个AI开发工程师. 你的名字是 小谷AI.', additional_kwargs={}, response_metadata={}), HumanMessage(content='你能开发哪些AI应用?', additional_kwargs={}, response_metadata={}), AIMessage(content='我能开发很多AI应用, 比如聊天机器人, 图像识别, 自然语言处理等.', additional_kwargs={}, response_metadata={}, tool_calls=[], invalid_tool_calls=[]), HumanMessage(content='你能帮我做什么?', additional_kwargs={}, response_metadata={})]
```

方式2：调用from_messages()--推荐

举例1：

```
In [14]: from langchain_core.prompts import ChatPromptTemplate
```

```
#创建实例
chat_prompt_template = ChatPromptTemplate.from_messages([
    ("system", "你是一个AI助手. 你的名字是 {name}.",),
    ("human", "我的问题是:{question}"),
])

response = chat_prompt_template.invoke(
    {"name": "小谷AI", "question": "如何学习人工智能?"}
)
print(response)
print(type(response))
print(len(response.messages)) #包含两条消息
```

```
messages=[SystemMessage(content='你是一个AI助手. 你的名字是 小谷AI.', additional_kwargs={}, response_metadata={}), HumanMessage(content='我的问题是:如何学习人工智能?', additional_kwargs={}, response_metadata={})]
<class 'langchain_core.prompt_values.ChatPromptValue'>
2
```

(2) 调用提示词模板的几种方法

- invoke()
- format()
- format_messages()
- format_prompt()

方法1：invoke() ---返回ChatPromptValue

传入的字典，返回ChatPromptValue 给大模型

```
In [15]: from langchain_core.prompts import ChatPromptTemplate
```

```
#创建实例
chat_prompt_template = ChatPromptTemplate.from_messages([
    ("system", "你是一个AI助手. 你的名字是 {name}.",),
    ("human", "我的问题是:{question}"),
])
```

```

response = chat_prompt_template.invoke(
    {"name": "小谷AI", "question": "如何学习人工智能?"} #invoke() 方法传入字典
)
print(response)
print(type(response))
print(len(response.messages)) #包含两条消息

```

```

messages=[SystemMessage(content='你是一个AI助手. 你的名字是 小谷AI.', additional_kwargs={}, response_metadata={}), HumanMessage(content='我的问题是:如何学习人工智能?', additional_kwargs={}, response_metadata={})]
<class 'langchain_core.prompt_values.ChatPromptValue'>
2

```

方法2：format() ---返回str

传入变量的值，返回str

In [16]: **from** langchain_core.prompts **import** ChatPromptTemplate

```

#创建实例
chat_prompt_template = ChatPromptTemplate.from_messages([
    ("system", "你是一个AI助手. 你的名字是 {name}.",),
    ("human", "我的问题是:{question}"),
])

response = chat_prompt_template.format(name="小谷AI",question="如何学习人工智能") #format
print(response)
print(type(response))

```

```

System: 你是一个AI助手. 你的名字是 小谷AI.
Human: 我的问题是:如何学习人工智能?
<class 'str'>

```

方法3：format_messages() ---返回消息构成的list

传入变量的值，返回消息构成的list

In [17]: **from** langchain_core.prompts **import** ChatPromptTemplate

```

#创建实例
chat_prompt_template = ChatPromptTemplate.from_messages([
    ("system", "你是一个AI助手. 你的名字是 {name}.",),
    ("human", "我的问题是:{question}"),
])

response = chat_prompt_template.format_messages(name="小谷AI",question="如何学习人工智能")
print(response)
print(type(response))

```

```

[SystemMessage(content='你是一个AI助手. 你的名字是 小谷AI.', additional_kwargs={}, response_metadata={}), HumanMessage(content='我的问题是:如何学习人工智能?', additional_kwargs={}, response_metadata={})]
<class 'list'>

```

方法4：format_prompt() ---返回ChatPromptValue

传入变量的值，返回ChatPromptValue

```
In [18]: from langchain_core.prompts import ChatPromptTemplate

#创建实例
chat_prompt_template = ChatPromptTemplate.from_messages([
    ("system", "你是一个AI助手. 你的名字是 {name}.",),
    ("human", "我的问题是:{question}"),
])

response = chat_prompt_template.format_prompt(name="小谷AI",question="如何学习人工智能?",)
print(response)
print(type(response))

messages=[SystemMessage(content='你是一个AI助手. 你的名字是 小谷AI.', additional_kwargs={}, response_metadata={}), HumanMessage(content='我的问题是:如何学习人工智能?', additional_kwargs={}, response_metadata={})]
<class 'langchain_core.prompt_values.ChatPromptValue'>
```

总结：调用提示词模板的几种方法

方法1：invoke() 传入的字典{}，返回ChatPromptValue

方法2：format() 传入变量的值，返回str

方法3：format_messages() 传入变量的值，返回消息构成的list

方法4：format_prompt() 传入变量的值，返回ChatPromptValue

如何实现ChatPromptValue与list[messages]、字符串之间转换？

```
In [23]: from langchain_core.prompts import ChatPromptTemplate

#创建实例
chat_prompt_template = ChatPromptTemplate.from_messages([
    ("system", "你是一个AI助手. 你的名字是 {name}.",),
    ("human", "我的问题是:{question}"),
])

response = chat_prompt_template.invoke(
    {"name": "小谷AI", "question": "如何学习人工智能?"}) #invoke() 方法传入字典
# response = chat_prompt_template.format_prompt(
# name="小谷AI", question="如何学习人工智能?" ) #format_prompt() 方法传入变量值
print(response)
print(f"invoke()方法输出为：{type(response)}")

#将ChatPromptValue转换为消息列表
response_messages = response.to_messages()
print(f"\n{response_messages}")
print(f"这里用to_messages转换为：{type(response_messages)}")

#将ChatPromptValue转换为字符串
response_to_string = response.to_string()
print(f"\n{response_to_string}")
print(f"这里用to_string转换为:{type(response_to_string)}")
```

```
messages=[SystemMessage(content='你是一个AI助手. 你的名字是 小谷AI.', additional_kwargs={}, response_metadata={}), HumanMessage(content='我的问题是:如何学习人工智能?', additional_kwargs={}, response_metadata={})]
invoke()方法输出为: <class 'langchain_core.prompt_values.ChatPromptValue'>
```

```
[SystemMessage(content='你是一个AI助手. 你的名字是 小谷AI.', additional_kwargs={}, response_metadata={}), HumanMessage(content='我的问题是:如何学习人工智能?', additional_kwargs={}, response_metadata={})]
这里用to_messages转换为: <class 'list'>
```

System: 你是一个AI助手. 你的名字是 小谷AI.

Human: 我的问题是:如何学习人工智能?

这里用to_string转换为:<class 'str'>

(3) 更丰富的实例化参数类型

不管用实例方法还是from_message()方式创建ChatPromptTemplate的实例，本质上来讲，传入的都是消息构成的列表。

调用上来讲，两种方法创建的ChatPromptTemplate的实例，messages参数类型都是列表，但是列表元素的类型是多样的，可以是：

- 字符串类型
- 字典类型
- 消息类型
- 元组构成的列表（最常用，最简单，最基础）
- Chat提示词模板类型
- 消息提示词模板类型

举例1：元组列表

```
In [24]: from langchain_core.prompts import ChatPromptTemplate
```

```
#创建实例
```

```
#第一种方式 构造
```

```
chat_prompt_template = ChatPromptTemplate(
    messages=[
        ("system", "你是一个AI助手. 你的名字是 {name}.",),
        ("human", "我的问题是:{question}"),
    ],
    #input_variables=["name", "question"] # 可选参数, 自动从消息中提取
)
```

```
#第二种方式 from_messages
```

```
chat_prompt_template = ChatPromptTemplate.from_messages([
    ("system", "你是一个AI助手. 你的名字是 {name}.",),
    ("human", "我的问题是:{question}"),
])
```

```

response = chat_prompt_template.invoke(
    {"name": "小谷AI", "question": "如何学习人工智能?"}) #invoke() 方法传入字典
# response = chat_prompt_template.format_prompt(
#     name="小谷AI", question="如何学习人工智能?" ) #format_prompt() 方法传入变量值
print(response)
print(type(response))

```

```

messages=[SystemMessage(content='你是一个AI助手. 你的名字是 小谷AI.', additional_kwargs={}, response_metadata={}), HumanMessage(content='我的问题是:如何学习人工智能?', additional_kwargs={}, response_metadata={})]
<class 'langchain_core.prompt_values.ChatPromptValue'>

```

举例2：字符串列表

In [25]: **from** langchain_core.prompts **import** ChatPromptTemplate

```

#创建实例
chat_prompt_template = ChatPromptTemplate.from_messages([
    "我的问题是{question}" #默认的角色是human
])

response = chat_prompt_template.invoke(
    {"question": "如何学习人工智能?"}) #invoke() 方法传入字典
# response = chat_prompt_template.format_prompt(
#     name="小谷AI", question="如何学习人工智能?" ) #format_prompt() 方法传入变量值
print(response)
print(type(response))

```

```

messages=[HumanMessage(content='我的问题是如何学习人工智能?', additional_kwargs={}, response_metadata={})]
<class 'langchain_core.prompt_values.ChatPromptValue'>

```

举例3：字典列表

In [26]: **from** langchain_core.prompts **import** ChatPromptTemplate

```

#创建实例
chat_prompt_template = ChatPromptTemplate.from_messages([
    {"role": "system", "content": "你是一个AI助手，你的名字是： {name}.",},
    {"role": "human", "content": "我的问题是： {question}" }
])

response = chat_prompt_template.invoke(
    {"name": "小智", "question": "如何学习人工智能?"}) #invoke() 方法传入字典
# response = chat_prompt_template.format_prompt(
#     name="小谷AI", question="如何学习人工智能?" ) #format_prompt() 方法传入变量值
print(response.to_messages())
print(response.to_string())
print(type(response))

```



```
[SystemMessage(content='你是一个AI助手，你的名字是：小智.', additional_kwargs={}, response_metadata={},), HumanMessage(content='我的问题是：如何学习人工智能?', additional_kwargs={}, response_metadata={})]
System: 你是一个AI助手，你的名字是：小智.
Human: 我的问题是：如何学习人工智能?
<class 'langchain_core.prompt_values.ChatPromptValue'>
```

举例4：消息对象列表

```
In [27]: from langchain_core.prompts import ChatPromptTemplate
from langchain_core.messages import SystemMessage, HumanMessage
#创建实例
chat_prompt_template = ChatPromptTemplate.from_messages([
    SystemMessage(content="你是一个AI助手，你的名字是小智"),
    HumanMessage(content="我的问题是：你好吗？")
])

response = chat_prompt_template.invoke({}) #invoke()方法传入消息对象列表
# response = chat_prompt_template.format_prompt() #format_prompt()方法传入变量
print(response.to_messages())
print(response.to_string())
print(type(response))
```

```
[SystemMessage(content='你是一个AI助手，你的名字是小智', additional_kwargs={}, response_metadata={},), HumanMessage(content='我的问题是：你好吗？', additional_kwargs={}, response_metadata={})]
System: 你是一个AI助手，你的名字是小智
Human: 我的问题是：你好吗？
<class 'langchain_core.prompt_values.ChatPromptValue'>
```

消息对象中是否可以声明变量？

不可以！ 并不会识别为变量

```
In [28]: from langchain_core.prompts import ChatPromptTemplate
from langchain_core.messages import SystemMessage, HumanMessage
#创建实例
chat_prompt_template = ChatPromptTemplate.from_messages([
    SystemMessage(content="你是一个AI助手，你的名字是：{name}.",),
    HumanMessage(content="我的问题是：{question}")
])

response = chat_prompt_template.invoke(
    {"name": "小智", "question": "如何学习人工智能?"}) #invoke()方法传入字典
# response = chat_prompt_template.format_prompt(
#     name="小谷AI", question="如何学习人工智能?" ) #format_prompt()方法传入变量值
print(response.to_messages())
print(response.to_string())
print(type(response))
```

```
[SystemMessage(content='你是一个AI助手，你的名字是：{name}.', additional_kwargs={}, response_metadata={},), HumanMessage(content='我的问题是：{question}', additional_kwargs={}, response_metadata={})]
System: 你是一个AI助手，你的名字是：{name}.
Human: 我的问题是：{question}
<class 'langchain_core.prompt_values.ChatPromptValue'>
```

举例5：BaseChatPromptTemplate参数列表

```
In [29]: from langchain_core.prompts import ChatPromptTemplate

#使用BaseChatPromptTemplate (嵌套的ChatPromptTemplate)
nested_prompt_template1 = ChatPromptTemplate.from_messages([
    ("system", "你是一个AI助手. 你的名字是 {name}.")
])
nested_prompt_template2 = ChatPromptTemplate.from_messages([
    ("human", "我的问题是:{question}"),
])

prompt_template = ChatPromptTemplate.from_messages([
    nested_prompt_template1,
    nested_prompt_template2,
])

prompt = prompt_template.format_messages(name="小谷AI",question="如何学习人工智能?")
print(prompt)
print(type(prompt))

print(prompt[0].content) #第一条消息
print(prompt[1].content) #第二条消息
```

```
[SystemMessage(content='你是一个AI助手. 你的名字是 小谷AI.', additional_kwargs={}, response_m
etadata={}), HumanMessage(content='我的问题是:如何学习人工智能?', additional_kwargs={}, resp
onse_metadata={})]
<class 'list'>
你是一个AI助手. 你的名字是 小谷AI.
我的问题是:如何学习人工智能?
```

举例6：BaseMessagePromptTemplate参数列表

```
In [30]: # 导入聊天消息类模板
from langchain_core.prompts import ChatPromptTemplate, HumanMessagePromptTemplate, Sys

# 创建消息模板
system_template = "你是一个AI助手，你的名字是：{name}."
system_message_prompt = SystemMessagePromptTemplate.from_template(system_template)

human_template = "我的问题是：{question}"
human_message_prompt = HumanMessagePromptTemplate.from_template(human_template)

# 组合成聊天提示模板
chat_prompt = ChatPromptTemplate.from_messages([system_message_prompt,
human_message_prompt])

# 格式化提示
formatted_messages = chat_prompt.format_messages(
    name="小智",
    question="如何学习人工智能？"
)
print(formatted_messages)
```

```
print(formatted_messages[0].content)
print(formatted_messages[1].content)
```

```
[SystemMessage(content='你是一个AI助手，你的名字是：小智.', additional_kwargs={}, response_
metadata={}), HumanMessage(content='我的问题是： 如何学习人工智能？', additional_kwargs={},
response_metadata={})]
```

你是一个AI助手，你的名字是：小智.

我的问题是： 如何学习人工智能？

与举例4的区别：消息对象列表中的变量没有被识别为变量

```
In [31]: from langchain_core.prompts import ChatPromptTemplate
from langchain_core.messages import SystemMessage, HumanMessage
#创建实例
chat_prompt_template = ChatPromptTemplate.from_messages([
    SystemMessage(content="你是一个AI助手，你的名字是： {name}."),
    HumanMessage(content="我的问题是： {question}")
])

response = chat_prompt_template.invoke({"name": "小智", "question": "如何学习人工智能?"}) #in
# response = chat_prompt_template.format_prompt(
# name="小谷AI", question="如何学习人工智能?" ) #format_prompt()方法传入变量值
print(response.to_messages())
print(response.to_string())
print(type(response))
```

```
[SystemMessage(content='你是一个AI助手，你的名字是： {name}.', additional_kwargs={}, respons
e_metadata={}), HumanMessage(content='我的问题是： {question}', additional_kwargs={}, respon
se_metadata={})]
```

System: 你是一个AI助手，你的名字是： {name}.

Human: 我的问题是： {question}

<class 'langchain_core.prompt_values.ChatPromptValue'>

综合案例

```
In [32]: # 示例1 使用 BaseMessage (已实例化消息)
system_msg = SystemMessage(content="你是一个AI工程师。")
human_msg = HumanMessage(content="你好！")

# 示例2 使用BaseMessagePromptTemplate
system_prompt = SystemMessagePromptTemplate.from_template("你是一个{role}")
human_prompt = HumanMessagePromptTemplate.from_template("{user_input}")

# 示例3 使用BaseChatPromptTemplate (嵌套的ChatPromptTemplate)
nested_prompt = ChatPromptTemplate.from_messages([("system", "嵌套提示词")])

prompt = ChatPromptTemplate.from_messages([
    system_msg,      #类型BaseMessage
    human_msg,       #类型BaseMessage
    system_prompt,    #类型BaseMessagePromptTemplate
    human_prompt,     #类型BaseMessagePromptTemplate
    nested_prompt    #类型BaseChatPromptTemplate
])

prompt.invoke({"role": "人工智能专家", "user_input": "一句话介绍一下大模型应用场景"})
```

```
Out[32]: ChatPromptValue(messages=[SystemMessage(content='你是一个AI工程师。', additional_kwargs={}, response_metadata={}), HumanMessage(content='你好！', additional_kwargs={}, response_metadata={}), SystemMessage(content='你是一个人工智能专家', additional_kwargs={}, response_metadata={}), HumanMessage(content='一句话介绍一下大模型应用场景', additional_kwargs={}, response_metadata={}), SystemMessage(content='嵌套提示词', additional_kwargs={}, response_metadata={})])
```

(4) 结合LLM

```
In [33]: # 提供大模型
from langchain.chat_models import init_chat_model
import os
from dotenv import load_dotenv

load_dotenv()

model = init_chat_model(
    model="deepseek:deepseek-v4-flash",
    api_key=os.getenv("DEEPSEEK_API_KEY"),
    base_url=os.getenv("DEEPSEEK_BASE_URL"),
    # extra_body={"thinking":{"type": "enabled"}}
)

# 通过Chat提示词模板, 创建提示词实例
from langchain_core.prompts import ChatPromptTemplate

# 创建实例
chat_prompt_template = ChatPromptTemplate.from_messages([
    ("system", "你是一个AI助手. 你的名字是 {name}.",),
    ("human", "我的问题是:{question}"),
])
```

```
prompt = chat_prompt_template.invoke(
    {"name": "小谷AI", "question": "一句话说明如何学习人工智能?"}
)

# 通过大模型调用提示词
response = model.invoke(prompt)
print(response.content)
```

学习人工智能最快的方式是“边做边学”：在掌握Python和数学基础后，直接动手实践项目，并对照教程反复迭代调试。

(5) ChatPromptTemplate的高级特性

1) 部分变量的预填充：partial()

使用场景

- 某些变量在所有调用中都相同
- 需要为不同用户/场景创建定制模板

In [34]: **from** langchain_core.prompts **import** ChatPromptTemplate

```
# 原始模板
template = ChatPromptTemplate.from_messages([
    ("system", "你是{role}，目标用户是{audience}"),
    ("user", "{task}")
])

result1 = template.invoke({"role": "导游", "audience": "游客", "task": "一句话介绍一下北京的故宫"})
result2 = template.invoke({"role": "导游", "audience": "游客", "task": "一句话介绍一下北京的颐和园"})

print(result1)
print(result2)
```

```
messages=[SystemMessage(content='你是导游，目标用户是游客', additional_kwargs={}, response
_metadata={}), HumanMessage(content='一句话介绍一下北京的故宫', additional_kwargs={}, respo
nse_metadata={})]
messages=[SystemMessage(content='你是导游，目标用户是游客', additional_kwargs={}, response
_metadata={}), HumanMessage(content='一句话介绍一下北京的颐和园', additional_kwargs={}, res
ponse_metadata={})]
```

使用partial()优化：

In [35]: **from** langchain_core.prompts **import** ChatPromptTemplate

```
# 原始模板
template = ChatPromptTemplate.from_messages([
    ("system", "你是{role}，目标用户是{audience}"),
    ("user", "{task}")
])
# 部分变量的预填充partial()
final_template = template.partial(role="导游", audience="游客")
```

```

result1 = fialal_template.invoke({"task":"一句话介绍一下北京的故宫"})
result2 = fialal_template.invoke({"task":"一句话介绍一下北京的颐和园"})

print(result1)
print(result2)

```

```

messages=[SystemMessage(content='你是导游，目标用户是游客', additional_kwargs={}, response_metadata={}), HumanMessage(content='一句话介绍一下北京的故宫', additional_kwargs={}, response_metadata={})]
messages=[SystemMessage(content='你是导游，目标用户是游客', additional_kwargs={}, response_metadata={}), HumanMessage(content='一句话介绍一下北京的颐和园', additional_kwargs={}, response_metadata={})]

```

In [36]: **from** langchain_core.prompts **import** ChatPromptTemplate

```

# 原始模板
template = ChatPromptTemplate.from_messages([
    ("system","你是{department}的{role}"),
    ("user","{task}")
])
# IT部门
it_template = template.partial(
    department = "IT部门",
    role = "技术支持"
)

# 销售部门
sales_template = template.partial(
    department = "销售部门",
    role = "销售顾问"
)
it_template.invoke({"task":"这个用什么技术架构？"})

```

Out[36]: ChatPromptValue(messages=[SystemMessage(content='你是IT部门的技术支持', additional_kwargs={}, response_metadata={}), HumanMessage(content='这个用什么技术架构？', additional_kwargs={}, response_metadata={})])

2) 消息占位符

JSON形式

In [37]: **from** langchain_core.prompts **import** ChatPromptTemplate

```

template = ChatPromptTemplate.from_messages(
    [
        ("system","你是一个有用的AI助手"),
        ("placeholder","{conversation}"),
    ]
)

prompt_value = template.invoke(
    {
        "conversation":[
            ("human","你好！"),
            ("ai","今天我能帮你做什么？"),
            ("human","你能给我做一个冰淇淋么？"),

```

```

        ("ai";"抱歉，我没有这样的能力")
    ]
}
)

print(prompt_value)

```

```

messages=[SystemMessage(content='你是一个有用的AI助手', additional_kwargs={}, response_metadata={}), HumanMessage(content='你好！', additional_kwargs={}, response_metadata={}), AIMessage(content='今天我能帮你做什么？', additional_kwargs={}, response_metadata={}, tool_calls=[], invalid_tool_calls=[]), HumanMessage(content='你能给我做一个冰淇淋么？', additional_kwargs={}, response_metadata={}), AIMessage(content='抱歉，我没有这样的能力', additional_kwargs={}, response_metadata={}, tool_calls=[], invalid_tool_calls=[])]

```

插入消息列表：MessagesPlaceholder

使用场景：当ChatPromptTemplate中的消息类型和个数不确定的时候，我们可以使用MessagesPlaceholder

举例1：

In [38]: **from** langchain_core.prompts **import** ChatPromptTemplate, MessagesPlaceholder

```

chat_prompt_template = ChatPromptTemplate.from_messages([
    ("system", "你是一个AI助手. 你的名字是 {name}.",
     MessagesPlaceholder(variable_name="msgs"), # 占位符
    ])

chat_prompt = chat_prompt_template.invoke(
    {
        "name": "小A",
        "msgs": [
            ("human", "你好！"),
            ("ai", "今天我能帮你做什么？"),
            ("human", "你能给我做一个冰淇淋么？"),
            ("ai", "抱歉，我没有这样的能力")
        ]
    }
)

# chat_prompt = chat_prompt_template.invoke({
#     "name": "小谷AI",
#     "msgs": [HumanMessage(content="如何学习人工智能?")]
# })
print(chat_prompt)

```

```

messages=[SystemMessage(content='你是一个AI助手. 你的名字是 小A.', additional_kwargs={}, response_metadata={}), HumanMessage(content='你好！', additional_kwargs={}, response_metadata={}), AIMessage(content='今天我能帮你做什么？', additional_kwargs={}, response_metadata={}, tool_calls=[], invalid_tool_calls=[]), HumanMessage(content='你能给我做一个冰淇淋么？', additional_kwargs={}, response_metadata={}), AIMessage(content='抱歉，我没有这样的能力', additional_kwargs={}, response_metadata={}, tool_calls=[], invalid_tool_calls=[])]

```

举例2：

```
In [39]: from langchain_core.prompts import ChatPromptTemplate, MessagesPlaceholder
from langchain_core.messages import HumanMessage, AIMessage

chat_prompt_template = ChatPromptTemplate.from_messages([
    ("system", "你是一个AI助手. 你的名字是 {name}.",
     MessagesPlaceholder(variable_name="msgs"), # 占位符
    ])

chat_prompt = chat_prompt_template.invoke({
    "name": "小谷AI",
    "msgs": [HumanMessage(content="如何学习人工智能?"),
             AIMessage(content="你可以从基础的数学和编程开始学习，然后逐步深入了解机器学习和深度学习")]
})
print(chat_prompt)
```

```
messages=[SystemMessage(content='你是一个AI助手. 你的名字是 小谷AI.', additional_kwargs={}, response_metadata={}), HumanMessage(content='如何学习人工智能?', additional_kwargs={}, response_metadata={}), AIMessage(content='你可以从基础的数学和编程开始学习，然后逐步深入了解机器学习和深度学习等领域。', additional_kwargs={}, response_metadata={}, tool_calls=[], invalid_tool_calls=[])]
```

3) 存储对话历史内容

```
In [40]: from langchain_core.prompts import ChatPromptTemplate, MessagesPlaceholder
from langchain_core.messages import HumanMessage, AIMessage

prompt = ChatPromptTemplate.from_messages([
    ("system", "你是一个AI助手. 你的名字是小智."),
    MessagesPlaceholder(variable_name="chat_history"), # 占位符
    ("human", "{question}")
])

prompt_value = prompt.format_messages(
    chat_history=[
        HumanMessage(content="如何学习人工智能?"),
        AIMessage(content="你可以从基础的数学和编程开始学习，然后逐步深入了解机器学习和深度学习"),
    ],
    question="我刚才的问题是什么?"
)

print(prompt_value)
```

```
[SystemMessage(content='你是一个AI助手. 你的名字是小智.', additional_kwargs={}, response_metadata={}), HumanMessage(content='如何学习人工智能?', additional_kwargs={}, response_metadata={}), AIMessage(content='你可以从基础的数学和编程开始学习，然后逐步深入了解机器学习和深度学习等领域。', additional_kwargs={}, response_metadata={}, tool_calls=[], invalid_tool_calls=[]), HumanMessage(content='我刚才的问题是什么?', additional_kwargs={}, response_metadata={})]
```

```
In [41]: from langchain_core.prompts import ChatPromptTemplate, MessagesPlaceholder
from langchain_core.messages import HumanMessage, AIMessage
```



```

prompt = ChatPromptTemplate.from_messages(
    [
        ("system", "你是一个AI助手. 你的名字是小智."),
        MessagesPlaceholder(variable_name="chat_history"), # 占位符
        ("human", "{question}")
    ]
)

prompt_value = prompt.invoke(
    {
        "chat_history":[
            ("human","5+2=?"),
            ("ai","5+2=7")
        ],
        "question":"我刚才的问题是什么？"
    }
)

print(prompt_value)

```

```

messages=[SystemMessage(content='你是一个AI助手. 你的名字是小智.', additional_kwargs={}, response_metadata={}), HumanMessage(content='5+2=?', additional_kwargs={}, response_metadata={}), AIMessage(content='5+2=7', additional_kwargs={}, response_metadata={}, tool_calls=[], invalid_tool_calls=[]), HumanMessage(content='我刚才的问题是什么？', additional_kwargs={}, response_metadata={})]

```

```

In [42]: from langchain.chat_models import init_chat_model
import os
import dotenv
dotenv.load_dotenv()

chat_model = init_chat_model(
    model="deepseek-v4-flash",
    model_provider="deepseek",
    api_key=os.getenv("DEEPSEEK_API_KEY")
)

chat_model.invoke(prompt_value).content

```

Out[42]: '你刚才问的是：“5+2=？” 我已经回答等于7啦。'

4) 可复用的模板库

实际项目中建议创建模板库

举例1：template.py文件声明模板库

```

In [:]: from langchain_core.prompts import ChatPromptTemplate
class PromptLibrary:
    """可复用的提示词模板库"""

    TRANSLATOR = ChatPromptTemplate.from_messages([
        ("system","你是专业翻译，精通{source_lang}和{target_lang}"),
        ("user","翻译以下文本：\n{text}")
    ])

```

```

CODE_REVIEWER = ChatPromptTemplate.from_messages([
    ("system","你是{language}代码审查专家，重点关注{foucs}"),
    ("user","审查代码：\n""{language}\n{code}\n""")
])

SUMMARIZER = ChatPromptTemplate.from_messages([
    ("system","你是内容摘要专家"),
    ("user","将以下内容总结为{num}个要点：\n{content}")
])

TUTOR = ChatPromptTemplate.from_messages([
    ("system","你是{subject}导师，学生水平：{level}"),
    ("user","{question}")
])

```

其他文件中使用：

```

In [ ]: from templates import PromptLibrary

messages = PromptLibrary.TRANSLATOR.format_messages(
    source_lang="英语",
    target_lang="中文",
    text="Hello World"
)

print(messages)

```

举例2：分类创建模板

```

templates/
|—— _init_.py
|—— common.py # 通用模板
|—— translation.py # 翻译相关模板
|—— coding.py # 编程相关

```

```

In [ ]: # common.py
from langchain_core.prompts import ChatPromptTemplate

FRIENDLY_ASSISTANT = ChatPromptTemplate.from_messages([
    ("system","你是一个友好的助手"),
    ("user","{input}")
])

```

5) 模板组合

将多个模板片段组合成复杂的提示词

方法1：字符串组合

```

In [45]: # 定义可复用的部分
role_part = "你是一个{domain}专家。"
style_part = "回答风格：{style}."
constraint_part = "限制：{sonstraint}."

```

```
# 组合
full_system = role_part + style_part + constraint_part

template = ChatPromptTemplate.from_messages([
    ("system", full_system),
    ("user", "{question}")
])
print(template)
```

```
input_variables=['domain', 'question', 'sonstraint', 'style'] input_types={} partial_variables={} messages=[SystemMessagePromptTemplate(prompt=PromptTemplate(input_variables=['domain', 'sonstraint', 'style'], input_types={}, partial_variables={}, template='你是一个{domain}专家。回答风格：{style}。限制：{sonstraint}。'), additional_kwargs={}), HumanMessagePromptTemplate(prompt=PromptTemplate(input_variables=['question'], input_types={}, partial_variables={}, template='{question}'), additional_kwargs={})]
```

方法2：使用+运算符

```
In [44]: from langchain_core.prompts import ChatPromptTemplate
template1 = ChatPromptTemplate.from_messages([
    ("system", "你是助手")
])

template2 = ChatPromptTemplate.from_messages([
    ("user", "{input}")
])

# 组合 (langchain 1.0支持)
combined = template1 + template2
print(combined)
```

```
input_variables=['input'] input_types={} partial_variables={} messages=[SystemMessagePromptTemplate(prompt=PromptTemplate(input_variables=[], input_types={}, partial_variables={}, template='你是助手'), additional_kwargs={}), HumanMessagePromptTemplate(prompt=PromptTemplate(input_variables=['input'], input_types={}, partial_variables={}, template='{input}'), additional_kwargs={})]
```

1.3 少量样本示例的提示词模板

1) 使用说明

FewShotPromptTemplate：与PromptTemplate一起使用

FewShotChatMessagePromptTemplate：与ChatPromptTemplate一起使用

Example selectors（示例选择器）：

2) FewShotPromptTemplate使用

举例1 未提供示例的情况：

LLM无法知道🐦代表什么意思

```
In [1]: import os
import dotenv
from langchain_openai import ChatOpenAI
dotenv.load_dotenv()
os.environ['OPENAI_API_KEY'] = os.getenv("DASHSCOPE_API_KEY")
os.environ['OPENAI_BASE_URL'] = os.getenv("DASHSCOPE_BASE_URL")

chat_model = ChatOpenAI(model="deepseek-v4-pro",
                        temperature=0.4)

res = chat_model.invoke("2 🐦 9是多少?")
print(res.content)
```

根据常见的网络表情符号谐音谜语，🐦 代表“鹦鹉”，其中“鹉”谐音数字“五”（5）。因此，“2 🐦 9”就是数字 2、5、9 的组合，即 259。

举例2 使用FewShotPromptTemplate

```
In [2]: from langchain_core.prompts import PromptTemplate, FewShotPromptTemplate
# 创建PromptTemplate实例
example_prompt = PromptTemplate.from_template(
    template="input: {input}\noutput: {output}"
)

# 提供一系列示例
examples = [
    {"input": "北京今天天气怎么样", "output": "北京市"},
    {"input": "南京下雨么", "output": "南京市"},
    {"input": "武汉热吗", "output": "武汉市"},
]

# 创建FewShotPromptTemplate实例
few_shot_template = FewShotPromptTemplate(
    example_prompt=example_prompt,
    examples=examples,
    suffix="input: {input}\noutput: ",
    input_variables=["input"]
)

few_shot_template.invoke({"input": "上海今天气温多少"})
```

```
Out[2]: StringPromptValue(text='input: 北京今天天气怎么样\noutput: 北京市\n\ninput: 南京下雨么\nout
put: 南京市\n\ninput: 武汉热吗\noutput: 武汉市\n\ninput: 上海今天气温多少\noutput: ')

```

调用大模型以后：

```
In [3]: chat_model.invoke(few_shot_template.invoke({"input": "上海今天气温多少"})).content
```

```
Out[3]: '上海市'
```

举例3：

```

In [4]: # 创建提示词模板
from langchain_core.prompts import FewShotPromptTemplate, PromptTemplate

# 创建提示词模板，配置一个提示词模板，将一个示例格式化为字符串
prompt_template = "你是一个数学专家，算式：input: {input}，值：output: {output}，使用：{description}"

# 这是一个提示词模板，用于设置每个示例的格式
prompt_sample = PromptTemplate.from_template(prompt_template)

# 提供示例
examples = [
    {"input": "2 + 2", "output": "4", "description": "加法"},
    {"input": "3 * 5", "output": "15", "description": "乘法"},
    {"input": "10 - 4", "output": "6", "description": "减法"},
]

# 创建FewShotPromptTemplate实例
prompt = FewShotPromptTemplate(
    example_prompt=prompt_sample,
    examples=examples,
    suffix="你是一个数学专家，算式：input: {input}，值：output: ",
    input_variables=["input", "output"]
)

print(prompt.invoke({"input": "8 / 2", "output": ""}))

# 调用大模型
response = chat_model.invoke(prompt.invoke({"input": "8 / 2", "output": ""}))
print(response.content)

```

text='你是一个数学专家，算式：input: 2 + 2，值：output: 4，使用：加法\n\n你是一个数学专家，算式：input: 3 * 5，值：output: 15，使用：乘法\n\n你是一个数学专家，算式：input: 10 - 4，值：output: 6，使用：减法\n\n你是一个数学专家，算式：input: 8 / 2，值：output: ' 你是一个数学专家，算式：input: 8 / 2，值：output: 4，使用：除法'

3) FewshotChatMessagePromptTemplate的使用

举例1：

```

In [5]: from langchain_core.prompts import FewShotChatMessagePromptTemplate, ChatPromptTemplate

# 1、示例消息格式
examples=[
    {"input": "1+1等于几？", "output": "1+1等于2"},
    {"input": "法国的首都都是？", "output": "巴黎"},
]

# 2、定义示例的消息格式提示词模板
msg_example_prompt = ChatPromptTemplate.from_messages([
    ("human", "{input}"),
    ("ai", "{output}"),
])

```

```
# 3、定义FewShotChatMessagePromptTemplate对象
few_shot_chat_prompt = FewShotChatMessagePromptTemplate(
    example_prompt=msg_example_prompt,
    examples=examples,
)

# 4、输出格式化后的消息
print(few_shot_chat_prompt.invoke({}))
```

```
messages=[HumanMessage(content='1+1等于几？', additional_kwargs={}, response_metadata={}),
AIMessage(content='1+1等于2', additional_kwargs={}, response_metadata={}, tool_calls=[], invalid_tool_calls=[]),
HumanMessage(content='法国的首都是？', additional_kwargs={}, response_metadata={}),
AIMessage(content='巴黎', additional_kwargs={}, response_metadata={}, tool_calls=[], invalid_tool_calls=[])]
```

举例2：

使用方式：将原始输入和被选中的示例组一起加入Chat提示词模板中

```
In [6]: # 1、导入相关包
from langchain_core.prompts import (FewShotChatMessagePromptTemplate,
ChatPromptTemplate)

# 2、定义示例组
examples = [
    {"input": "2 🦉 2", "output": "4"},
    {"input": "2 🦉 3", "output": "8"},
]

# 3、定义示例的消息格式提示词模板
example_prompt = ChatPromptTemplate.from_messages([
    ("human", "计算：{input}"),
    ("ai", "结果是：{output}"),
])

# 4、定义FewShotChatMessagePromptTemplate对象
few_shot_prompt = FewShotChatMessagePromptTemplate(
    example_prompt=example_prompt, # 示例提示词模板
    examples=examples, # 示例组
)

# 5、输出完整提示词的消息模板
final_prompt = ChatPromptTemplate.from_messages(
    [
        ("system", "你是一个数学专家。"),
        few_shot_prompt, # 嵌入FewShotChatMessagePromptTemplate对象
        ("human", "计算：{input}"),
    ]
)

# 6、提供大模型
import os
import dotenv
from langchain.chat_models import init_chat_model
dotenv.load_dotenv()
```

```

chat_model = init_chat_model(model="openai:deepseek-v4-pro",
                             api_key=os.getenv("DASHSCOPE_API_KEY"),
                             base_url=os.getenv("DASHSCOPE_BASE_URL"),
                             temperature=0.4)

# 7、调用大模型
response = chat_model.invoke(
    final_prompt.invoke({"input": "2 🐦 4"})
).content
print(response)

```

2⁴ = 16

4) Example selectors(示例选择器)

避免盲目传递所有示例，减少 token 消耗的同时，还可以提升输出效果。

- **语义相似选择**：通过余弦相似度等度量方式评估语义相关性，选择与输入问题最相似的 k 个示例。
- **长度选择**：根据输入文本的长度，从候选示例中筛选出长度最匹配的示例。增强模型对文本结构的理解。比语义相似度计算更轻量，适合对响应速度要求高的场景。
- **最大边际相关示例选择**：优先选择与输入问题语义相似的示例；同时，通过惩罚机制避免返回同质化的内容

举例1：

```

In [7]: # 1. 导入相关包
from langchain_community.vectorstores import Chroma
from langchain_core.example_selectors import SemanticSimilarityExampleSelector
import os
import dotenv
from langchain_community.embeddings import DashScopeEmbeddings
dotenv.load_dotenv()

# 2. 定义嵌入模型
os.environ['OPENAI_API_KEY'] = os.getenv("DASHSCOPE_API_KEY")
os.environ['OPENAI_BASE_URL'] = os.getenv("DASHSCOPE_BASE_URL")

#####
embeddings_model = DashScopeEmbeddings(
    model="qwen3.7-text-embedding"
)

# 3. 定义示例组
examples = [
    {
        "question": "谁活得更久，穆罕默德·阿里还是艾伦·图灵?",
        "answer": ""
        接下来还需要问什么问题吗？
        追问：穆罕默德·阿里去世时多大年纪？
        中间答案：穆罕默德·阿里去世时享年74岁。
        """,
    },

```

```

    },
    {
        "question": "craigslist的创始人是什么时候出生的?",
        "answer": ""
        接下来还需要问什么问题吗?
        追问: 谁是craigslist的创始人?
        中级答案: Craigslist是由克雷格·纽马克创立的。
        "",
    },
    {
        "question": "谁是乔治·华盛顿的外祖父?",
        "answer": ""
        接下来还需要问什么问题吗?
        追问: 谁是乔治·华盛顿的母亲?
        中间答案: 乔治·华盛顿的母亲是玛丽·鲍尔·华盛顿。
        "",
    },
    {
        "question": "《大白鲨》和《皇家赌场》的导演都来自同一个国家吗?",
        "answer": ""
        接下来还需要问什么问题吗?
        追问: 《大白鲨》的导演是谁?
        中级答案: 《大白鲨》的导演是史蒂文·斯皮尔伯格。
        "",
    },
]
# 4. 定义示例选择器
example_selector = SemanticSimilarityExampleSelector.from_examples(
    # 这是可供选择的示例列表
    examples,
    # 这是用于生成嵌入的嵌入类, 用于衡量语义相似性
    embeddings_model,
    # 这是用于存储嵌入并进行相似性搜索的 VectorStore 类
    Chroma,
    # 这是要生成的示例数量
    k=1,
)
# 选择与输入最相似的示例

question = "玛丽·鲍尔·华盛顿的父亲是谁?"
selected_examples = example_selector.select_examples({"question": question})
print(f"与输入最相似的示例: {selected_examples}")

for example in selected_examples:
    print("\n")
for k, v in example.items():
    print(f"{k}: {v}")

```


与输入最相似的示例：[{'answer': '\n 接下来还需要问什么问题吗？\n 追问：谁是乔治·华盛顿的母亲？\n 中间答案：乔治·华盛顿的母亲是玛丽·鲍尔·华盛顿。', 'question': '谁是乔治·华盛顿的外祖父？'}]

answer:

接下来还需要问什么问题吗？

追问：谁是乔治·华盛顿的母亲？

中间答案：乔治·华盛顿的母亲是玛丽·鲍尔·华盛顿。

question: 谁是乔治·华盛顿的外祖父？

举例2：结合FewShotPromptTemplate使用

这里使用FAISS，需要安装，`pip install faiss-cpu`

```
In [8]: #1. 导入相关包
from langchain_community.vectorstores import FAISS
from langchain_core.example_selectors import SemanticSimilarityExampleSelector
from langchain_core.prompts import FewShotPromptTemplate, PromptTemplate
from langchain_openai import OpenAIEmbeddings

# 2. 定义示例提示词模版
example_prompt = PromptTemplate.from_template(
    template="Input: {input}\nOutput: {output}",
)

# 3. 创建一个示例提示词模版
examples = [
    {"input": "高兴", "output": "悲伤"},
    {"input": "高", "output": "矮"},
    {"input": "长", "output": "短"},
    {"input": "精力充沛", "output": "无精打采"},
    {"input": "阳光", "output": "阴暗"},
    {"input": "粗糙", "output": "光滑"},
    {"input": "干燥", "output": "潮湿"},
    {"input": "富裕", "output": "贫穷"},
]

# 4. 定义嵌入模型
os.environ['OPENAI_API_KEY'] = os.getenv("SILICON_API_KEY")
os.environ['OPENAI_API_BASE'] = os.getenv("SILICON_BASE_URL")

#####
embeddings = OpenAIEmbeddings(
    model="Qwen/Qwen3-Embedding-8B",
)

# 5. 创建语义相似性示例选择器
example_selector = SemanticSimilarityExampleSelector.from_examples(
    examples,
    embeddings,
    FAISS,
    k=2,
)

#或者
#example_selector = SemanticSimilarityExampleSelector(
# examples,
```

```
# embeddings,
# FAISS,
# k=2
#)

# 6. 定义小样本提示词模版
similar_prompt = FewShotPromptTemplate(
    example_selector=example_selector,
    example_prompt=example_prompt,
    prefix="给出每个词组的反义词",
    suffix="Input: {word}\nOutput:",
    input_variables=["word"],
)
response = similar_prompt.invoke({"word": "忧郁"})
print(response.text)
```

给出每个词组的反义词

Input: 精力充沛

Output: 无精打采

Input: 粗糙

Output: 光滑

Input: 忧郁

Output:

调用LLM

```
In [9]: import os
import dotenv
from langchain.chat_models import init_chat_model
dotenv.load_dotenv()

chat_model = init_chat_model(model="openai:deepseek-v4-pro",
                             api_key=os.getenv("DASHSCOPE_API_KEY"),
                             base_url=os.getenv("DASHSCOPE_BASE_URL"),
                             temperature=0.4)
res = chat_model.invoke(response)
print(res.content)
```

开朗

5) PipelinePromptTemplate(了解)

用于将多个提示模板按顺序组合成处理管道，实现分阶段、模块化的提示构建。它的核心作用类似于软件开发中的管道模式（Pipeline Pattern），通过串联多个提示处理步骤，实现复杂的提示生成逻辑。

特点：将复杂提示拆解为多个处理阶段，每个阶段使用独立的提示模板前一个模板的输出作为下一个模板的输入变量

使用场景：解决单一超大提示模板难以维护的问题

说明：PipelinePromptTemplate在langchain 0.3.22版本中被标记为过时，在langchain-core==1.0之前不会删除它。

```
In []: from langchain_core.prompts.prompt import PromptTemplate

# 阶段1：问题分析
analysis_template = PromptTemplate.from_template("""
    分析这个问题：{question}
    关键要素：
    """)

# 阶段2：知识检索
retrieval_template = PromptTemplate.from_template("""
    基于以下要素搜索资料：
    {analysis_result}
    搜索关键词：
    """)

# 阶段3：生成最终回答
answer_template = PromptTemplate.from_template("""
    综合以下信息回答问题：
    {retrieval_result}
    最终答案：
    """)

# 逐步执行管道提示
pipeline_prompts = [
    ("analysis_result", analysis_template),
    ("retrieval_result", retrieval_template)
]

my_input = {"question": "量子计算的优势是什么?"}

print(pipeline_prompts)
# [('analysis_result', PromptTemplate(input_variables=['question'], input_types={}, partial_variables={}, template='\n分析这个问题：{question}\n关键要素：\n'))], (
# PromptTemplate(input_variables=['analysis_result'], input_types={}, partial_variables={}, template='\n基于以下要素搜索资料：\n{analysis_result}\n搜索关键词：\n')))]

for name, prompt in pipeline_prompts:
    # 调用当前提示模板并获取字符串结果
    result = prompt.invoke(my_input).to_string()
    # 将结果添加到输入字典中供下一步使用
    my_input[name] = result

# 生成最终答案
my_output = answer_template.invoke(my_input).to_string()
print(my_output)
```

6) 自定义提示词模版(了解)

在创建prompt时，我们也可以按照自己的需求去创建自定义的提示模版。

步骤：

自定义类继承提示词基类模版BasePromptTemplate

重写format、format_prompt、from_template方法

```
In [10]: # 1. 导入相关包
from typing import List, Dict, Any
from langchain_core.prompts import BasePromptTemplate
from langchain_core.prompts import PromptTemplate
from langchain_core.prompt_values import PromptValue

# 2. 自定义提示词模版
class SimpleCustomPrompt(BasePromptTemplate):
    """简单自定义提示词模板"""
    template: str # type: ignore

    def __init__(self, template: str, **kwargs): # type: ignore
        # 使用PromptTemplate解析输入变量
        # prompt = PromptTemplate.from_template(template)
        #
        #
        super().__init__(
            input_variables=prompt.input_variables,
            template=template,
            **kwargs
        )

    def format(self, **kwargs: Any) -> str: # type: ignore
        """格式化提示词"""
        # print("kwargs:", kwargs)
        # print("self.template:", self.template)

        return self.template.format(**kwargs)

    def format_prompt(self, **kwargs: Any) -> PromptValue:
        """实现抽象方法"""
        return PromptValue(text=self.format(**kwargs))

    @classmethod
    def from_template(cls, template: str, **kwargs) -> "SimpleCustomPrompt": # type: ignore
        """从模板创建实例"""
        return cls(template=template, **kwargs)

# 3. 使用自定义提示词模版
custom_prompt = PromptTemplate.from_template(
    template="请回答关于{subject}的问题：{question}"
)
```

```
# 4. 格式化提示词
formatted = custom_prompt.format(
    subject="人工智能",
    question="什么是LLM?"
)

print(formatted)
```

请回答关于人工智能的问题：什么是LLM？



7) 从文档中加载Prompt(了解)

一方面，将想要设定prompt所支持的格式保存为JSON或者YAML格式文件。

另一方面，通过读取指定路径的格式化文件，获取相应的prompt。

目的与使用场景：

为了便于共享、存储和加强对prompt的版本控制。

当我们的prompt模板数据较大时，我们可以使用外部导入的方式进行管理和维护。

(1) yaml格式提示词

asset下创建yaml文件：prompt.yaml

yaml文件内写入：

```
In [ ]: _type:
        "prompt"
input_variables:
        ["name", "what"]
template:
        "请给{name}讲一个关于{what}的故事"
```

```
In [ ]: from langchain_core.prompts import load_prompt
        from dotenv import load_dotenv

        load_dotenv()

        prompt = load_prompt("asset/prompt.yaml", encoding="utf-8")

        print(prompt)

        print(prompt.format(name="年轻人", what="滑稽"))
```

(2) json格式提示词

Json文件中写入：

```
In [ ]: {
        "_type": "prompt",
```

```
"input_variables": ["name", "what"],  
"template": "请{name}讲一个{what}的故事。 "  
}
```

```
In [ ]: from langchain_core.prompts import load_prompt  
        from dotenv import load_dotenv  
  
        load_dotenv()  
  
        prompt = load_prompt("asset/prompt.json", encoding="utf-8")  
        print(prompt.format(name="张三", what="搞笑的"))
```

(3) 调用py文件提示词模板

详见ChatPromptTemplate高级特性--4) 可复用的模板库